When the Tests Pass but the Proof Fails: Verifier and Test Feedback for LLM Coding Agents in Dafny and Python

Syed Muhammad Hassan
Roll No. 23i-0519 (Group Head)
FAST-NUCES, Islamabad
i230519@isb.nu.edu.pk

Abdul Majeed
Roll No. 23p-3038
FAST-NUCES, Islamabad
p233038@isb.nu.edu.pk

AI4015 Agentic AI, Fall 2026
Project Code: **G1-XX**
Supervisor: Muhammad Sajjad
Code: https://github.com/syedhassan13/SpecForge

***Abstract*—Verifiable code generation asks a language model for code that a verifier can prove correct against a formal specification. The VERINA benchmark showed that this is hard in Lean, but it evaluated models in a single shot, in one language and without a test-driven baseline. We study what happens when the same tasks are given to an agent that receives feedback. We re-expressed 97 VERINA-BASIC tasks in Dafny and in Python with iContract contracts checked by CrossHair, and we checked that the two versions of each task are equivalent using shared tests, specification probes and differential testing. We then built SpecLoop, an agent whose feedback sources can be switched on and off, with a router that uses test results to separate wrong programs from missing proofs and a guard that rejects any change to the specification. With GPT-4o-mini and three seeds per configuration, VERINA's protocol passes 69.3% of tasks in Dafny and 98.7% in Python, against the 44.0% reported in Lean. On Dafny, testing-only feedback raises correctness from 46.8% to 56.3% but stops with an unverified program in 26.8% of runs, verifier feedback reaches 63.2%, and SpecLoop reaches 68.0%. Python is near the ceiling for every configuration. The results suggest that the choice of specification language decides most of the outcome, that a verifier is worth adding to a test-driven agent mainly because it tells the agent when it is not done, and that tests make verifier feedback easier to act on. All code, data, trajectories and model responses are released so that every number can be reproduced without API calls.**

***Index Terms*—LLM agents, verifiable code generation, formal verification, Dafny, design by contract, program repair**

I. Introduction

LLM coding agents are now good at producing code that looks right, and a short test suite usually agrees. The trouble is that tests only sample the input space. Errors made by LLMs tend to be semantic and spread over several lines [1], and a handful of tests can let them through. A formal specification checked by a verifier closes that gap in principle: if the program is proved against its postcondition, it is correct for every input that satisfies the precondition. This is the idea behind *verifiable code generation*, where a model has to produce code together with the proof that it meets a formal specification.

VERINA [2], our base paper, is the most complete benchmark we know of for this task. It gives 189 Lean 4 problems with descriptions, reference implementations, specifications and tests. It evaluates code, specification and proof generation separately and together. The proof results are weak: the best general model passes 72.6% of tasks on code generation but proves only 4.9%. On the 108 easier VERINA-BASIC tasks, GPT-4o-mini passes 44.0% and more than half of its programs do not even compile in Lean.

Three parts of VERINA's design leave questions open for anyone who wants to build such an agent. We fixed these as our research gaps in Phase 1 of the project. *Gap 1:* VERINA uses one language, Lean. Most failures are Lean compile errors, so it is unclear how much of the difficulty comes from verifiable code generation itself and how much from the language. No study compares specification languages on the same tasks under the same agent loop. *Gap 2:* VERINA evaluates code in a single shot. Its only loop is proof refinement driven by Lean compiler messages, and its tests are used for scoring, never as feedback. Agents that use verifier feedback (for example [3]–[5]) also treat testing as a separate measurement. Whether verifier and test feedback work better together than apart has not been measured. *Gap 3:* there is no testing-only baseline. A practitioner already has tests, so the useful question is what a verifier adds on top of them, and VERINA's design cannot answer it.

To answer them we ran a controlled study. We re-expressed VERINA-BASIC in Dafny [6] and in Python with iContract contracts checked by CrossHair [7], and kept 97 tasks whose two versions pass a set of equivalence checks. We then built SpecLoop, an agent whose feedback sources can be switched on and off one at a time. We ran it on both languages with three seeds under one fixed budget. Table I lists each gap together with the evidence for it, our approach and how we measure it.

Each gap becomes one research question (Section III). In short, we find the following. First, the specification language matters more than any feedback signal we tried. Under VERINA's own protocol and with the same model, the share of programs that pass all tests goes from 44.0% in Lean to 69.3% in Dafny and 98.7% in Python, mostly because far fewer programs fail to compile. Python stays ahead of Dafny in every agent configuration (98.7% against 68.0% for the full

TABLE I
RESEARCH GAPS (FIXED IN PHASE 1), SUPPORTING EVIDENCE, OUR APPROACH AND HOW EACH IS MEASURED.

Gap	Evidence in the literature	Our approach (component)	Measurement
G1: no controlled comparison of specification languages under one loop	VERINA and CLEVER use Lean only [2], [8]; DafnyBench and MBPP-DFY use Dafny only [5], [9]; Shefer et al. compare languages but without a test baseline [10]	Same tasks in Dafny and Python/iContract with probe-checked equivalent specs; diagnostic normaliser so one loop runs on both	Paired (by task) correctness, verified rate and first-attempt test pass rate; Wilcoxon and McNemar tests (E4)
G2: verifier and test feedback never combined	Verifier-driven repair in AlphaVerus, AutoVerus, DafnyBench, VERINA refinement [2]–[5]; test-driven repair in Self-Debug, Reflexion, LDB [11]–[13]	Test channel added to the verifier loop, and a router that uses test outcomes to tell a wrong program from a missing proof	Correctness of SpecLoop vs. verifier-only and testing-only (E4); router ablation (E5)
G3: no testing-only baseline	VERINA reports single-shot and proof-refinement numbers only [2]; no paper in our review compares a verifier loop with a test loop on the same agent	Verifier feedback channel, compared with an identical agent that only sees tests	Correctness gain of verifier-only over testing-only, and false-success rate of the testing-only agent (E1, E4)

agent). Second, the verifier adds real value over a test-driven loop: on Dafny, verifier-only feedback is +6.9 pp more correct than testing-only feedback, and the testing-only agent stops with an unverified program it believes is finished in 26.8% of its runs. Third, tests and verifier work best together. In 39.3% of failed Dafny verifications the program already passed every test, which means the code was right and only the proof was missing. SpecLoop uses the tests to recognise these cases and is the most accurate configuration (+4.8 pp over verifier-only), although the gain is smaller than we hypothesised.

Our contributions are: (1) SpecLoop-VB, 97 paired Dafny/Python versions of VERINA-BASIC tasks with verified references, admitted only after spec probes and differential testing; (2) an agent in which each feedback source and each support component can be removed independently, with an integrity guard that we stress-test against tampering; (3) a controlled evaluation of the three gaps, with repeated runs, paired statistics, ablations, a cost analysis and a failure analysis; and (4) all code, data, trajectories and an LLM response cache that reproduces every number in this paper without new API calls.

II. RELATED WORK

Benchmarks for verifiable code generation. VERINA [2] is the base paper of this work. It contains 189 Lean 4 tasks (108 in VERINA-BASIC, 81 in VERINA-ADV), each with a description, a reference implementation, a formal pre- and postcondition, and positive and negative tests. It scores code generation (CodeGen), specification generation (SpecGen) and proof generation (ProofGen) separately and in combination; o3, the best general model, reaches 72.6%, 52.3% and 4.9%. On VERINA-BASIC, GPT-4.1 reaches 69.8% CodeGen pass@1 and GPT-4o-mini 44.0%, and most failures are programs that do not compile in Lean (27.4% and 54.9% of samples). The authors also try iterative proof refinement with Lean compiler messages, which lifts ProofGen to at most 20.1% after 64 rounds. CLEVER [8] adapts 161 HumanEval problems to Lean. The model first writes a specification and proves it equivalent to a hidden reference specification, then writes an implementation and proves it correct. Its best configuration solves 1 of 161 problems end to end. DafnyBench [5] contains more than 750 Dafny programs and asks the model to fill in the hints (loop invariants, assertions) a program needs to verify. With ten attempts and verifier messages GPT-4o reaches 59.3% and Claude 3 Opus 67.8%. MBPP-DFY [9] and CloverBench [14] are the Dafny datasets that VERINA-BASIC was translated from. Misu et al. [9] report that GPT-4 produces verified Dafny methods for 58% of 178 MBPP problems, but only with a retrieval-augmented chain-of-thought prompt (10% with a signature-only prompt). All of these benchmarks use one verification language, so they cannot tell how much a result depends on that choice.

LLMs with program verifiers. AlphaVerus [3] translates Dafny programs into Verus, repairs them with a tree search guided by verifier errors (Treefinement) and filters out candidates that game the specification. It raises verified rates on HumanEval-Verus and MBPP-Verus without fine-tuning. AutoVerus [4] uses a team of LLM agents for Verus proofs and proves over 90% of the 150 tasks in Verus-Bench, and SAFE [15] improves Rust proof generation through self-evolution. Laurel [16] generates helper assertions for Dafny, Lemur [17] generates invariants with a verifier in the loop, Baldur [18] and COPRA [19] target interactive provers, and Chakraborty et al. [20] show that retrieval helps proof synthesis in F*. All of them use verifier feedback, but none compares it with test feedback for the same agent or runs the same loop in two specification languages.

Specifications and tests. SpecGen [21], AutoSpec [22] and nl2postcond [23] generate formal specifications or postconditions with LLMs. TOGLL [24] generates Java test oracles that kill 1,023 mutants EvoSuite misses, yet fewer mutants than EvoSuite overall, so generated oracles help but are not complete. Wang et al. [1] studied 557 incorrect HumanEval solutions from six LLMs and found that most errors span several lines of code and often need multi-hunk fixes, the kind of semantic error that a small test suite can let through. Shefer et al. [10] are, as far as we know, the closest to our Gap 1: they compare how well an LLM writes verified code in Dafny, Nagini (Python) and Verus, with up to five rounds of verifier feedback and a final check against the reference specification. They do not compare verifier feedback with test feedback, and

TABLE II
POSITIONING AGAINST THE CLOSEST WORK. FB = FEEDBACK SOURCE USED INSIDE THE LOOP (V = VERIFIER, C = COMPILER, T = TESTS).

Work	Languages	FB	Test-only baseline	Spec
VERINA [2]	Lean	C (proofs only)	no	sorry
CLEVER [8]	Lean	C	no	yes
DafnyBench [5]	Dafny	V	no	yes
AlphaVerus [3]	Verus	V	no	filter
AutoVerus [4]	Verus	V	no	filter
Shefer et al. [10]	Dafny, Py, Rust	V	no	yes
SpecLoop (ours)	Dafny, Py	V, T, V+T	yes	yes

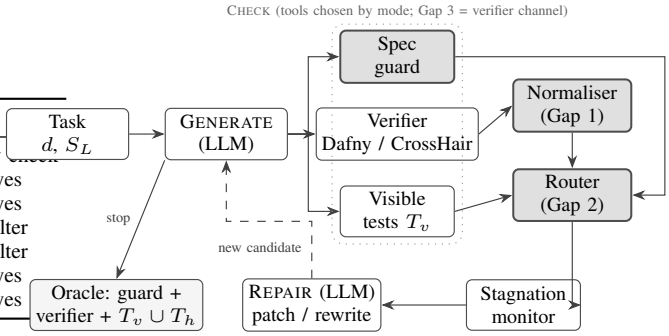


Fig. 1. SpecLoop architecture. Shaded boxes are the gap-specific components, each of which can be switched off. The oracle is outside the agent and judges every candidate in the same way.

they have no testing-only agent, which is the comparison our Gaps 2 and 3 need.

Feedback-driven coding agents. Self-Debugging [11], Reflexion [12] and LDB [13] improve code by feeding execution results back to the model, Self-Refine [25] uses the model's own critique, CodeT [26] uses generated tests to rank samples, and ReAct [27] and SWE-agent [28] interleave reasoning with tool calls. For code the external signal is almost always test execution, so our testing-only agent, a direct instance of this family, is the baseline VERINA lacks.

Table II summarises how our study differs from the closest work. What is new is the controlled design: one agent, two specification languages with probe-checked equivalent specifications, and four feedback configurations that differ in exactly one signal.

III. PROBLEM FORMULATION

A task is a tuple $t = (d, S_L, T)$ where d is a natural-language description, S_L is a specification written in language $L \in \{\text{DAFNY}, \text{PY}\}$, and T is a list of input/output tests. For Dafny, S_L is a method header with `requires` and `ensures` clauses plus any helper predicates; for Python it is a function signature decorated with iContract `require`/`ensure` conditions. In both cases the body is empty. We split T into a visible part T_v (the first $\lceil |T|/2 \rceil$ tests) and a held-out part T_h.

An agent receives d and S_L and must return a complete program P. It may call a set of tools $F \subseteq \{\textit{verifier}, \textit{tests}\}$ on intermediate candidates, where *tests* runs only T_v, and it may check at most $K = 5$ candidates. The four agent configurations we compare differ only in F: single-shot ($K = 1$), testing-only ($F = \{\textit{tests}\}$), verifier-only ($F = \{\textit{verifier}\}$), and SpecLoop ($F = \{\textit{verifier}, \textit{tests}\}$ plus the routing described in Section IV).

A final program is *correct* when three conditions hold together:

$$\text{Correct}(P) \iff \text{Intact}(P, S_L) \wedge \text{Verified}_L(P) \wedge \text{Pass}(P, T_v \cup T_h) \quad (1)$$

Intact is decided by the integrity guard (Section IV-E), Verified by `dafny verify` or by CrossHair, and Pass by running all tests. The first condition matters because a verifier only checks the specification that is in the file it receives. VERINA's CodeGen metric corresponds to Pass alone, and we report it separately for the base-paper comparison.

The three research questions follow the gaps fixed in Phase 1, with the Gap 3 question rewritten after supervisor feedback so that it matches the gap itself:

RQ1 (Gap 1). Under the same agent loop and the same tasks, does the specification language change how often the agent produces correct code? *H1a:* Dafny reaches a higher verification rate than Python/iContract with CrossHair. *H1b:* Python reaches a higher first-attempt test pass rate.

RQ2 (Gap 2). Does feedback that combines verifier and test results give higher correctness than either source alone? *H2:* SpecLoop is at least 5 percentage points (pp) more correct than the better of verifier-only and testing-only.

RQ3 (Gap 3). VERINA has no testing-only baseline, so it cannot say what a verifier adds beyond the tests an agent would normally use. How large is that gain? *H3:* the verifier-only agent is at least 10 pp more correct than the testing-only agent. As a secondary question we check whether the gain holds for more than one model.

IV. PROPOSED SYSTEM

Fig. 1 shows SpecLoop. It is a small state machine around an LLM: GENERATE produces a first candidate, CHECK calls tools, ROUTE turns tool output into feedback, and REPAIR asks for a new candidate. The loop stops when every check of the current configuration passes or after K candidates. Every model call uses structured output, a JSON object with an `analysis` field, a `strategy` field (`patch` or `rewrite`) and the full program, so the model's stated reasoning and repair decision are recorded with each step. All four configurations share the same system prompt and the same first prompt. Since model responses are cached by request, a given task, model and seed produce the same first candidate in every configuration, and differences between configurations come only from the feedback.

A. Tools and environment

The agent works against two real toolchains. For Dafny it calls `dafny verify` (Dafny 4.11 with its bundled Z3 [29], 20 s per verification condition) and, for tests, compiles the program together with a generated `Main` method to Python and runs it. For Python it runs CrossHair [7] in iContract

mode (6 s per condition) and executes the tests with contracts enabled, so a violated postcondition shows up as a failing test. Test cases are stored once in a language-neutral JSON form and rendered into both languages, so the two versions of a task see exactly the same inputs and expected outputs. Tool results are memoised by program hash, so resubmitting an identical program does not cost another verifier run.

B. Gap 1 component: diagnostic normaliser

Dafny and CrossHair report problems in very different shapes. Dafny gives a file position, a message such as "a postcondition could not be proved on this return path" and a separate "related location" that points at the clause. CrossHair reports the line of the violated contract and a concrete call. Neither output is easy for a model to use directly, because both refer to line numbers in a file the model never sees. The normaliser maps both into one taxonomy (`syntax`, `resolution`, `postcondition`, `invariant_entry`, `invariant_maintained`, `termination`, `index_bounds`, `counterexample`, `exception`, `timeout`), quotes the offending source line and the specification clause involved, and adds a one-line explanation of each kind. With this in place, one agent loop runs unchanged on both languages. With the component off, the model receives the raw tool output with temporary paths removed, which is what the Phase 1 prototype did.

C. Gap 2 component: feedback router

The router starts from one observation. A Dafny error such as "postcondition could not be proved" is ambiguous, since Dafny reports it both when the program is wrong and when the program is right but the loop invariants are missing. Tests can separate the two cases cheaply. The router therefore classifies every failed check: SPEC_TAMPER (the guard fired), SYNTAX (does not compile), LOGIC_BUG (a visible test fails or CrossHair produced a counterexample), and PROOF_GAP (all visible tests pass but verification fails). Each class comes with a different instruction. For a logic bug the model is told to fix the algorithm first and is shown failing tests before verifier messages. For a proof gap it is told to keep the algorithm and strengthen loop invariants, `decreases` clauses or assertions. For a compile error the model sees only the compile errors. With the router off, SpecLoop pastes the verifier output and the test output one after the other, as the Phase 1 combined agent did.

D. Gap 3 component: verifier feedback channel

Gap 3 is about the verifier signal itself, so its component is just the verifier channel in CHECK. Removing it turns SpecLoop into the testing-only agent, which is our E1 baseline and the configuration VERINA does not have.

E. Specification integrity guard

A program that edits its own specification can be "verified" and still be wrong. The guard compares each candidate with the original specification before any tool runs. For Dafny it strips comments, normalises whitespace and requires the target method header (signature and every `requires`/`ensures` clause) and every other declaration from the specification file to appear unchanged. It also rejects `assume`, `{:axiom}`, `{:verify false}`, `{:only}`, `{:extern}` and `decreases *`, and any Dafny warning about bodyless lemmas. For Python it compares abstract syntax trees, so formatting does not matter, and requires identical decorators, signature, imports and helper definitions. CrossHair directive comments, assignments to the `icontract` module, `eval`/`exec` and access to `sys.modules` are rejected as well. A rejected candidate is not verified at all. The model is told exactly what it changed and has to restore it. Even when the in-loop guard is switched off, the oracle applies the same guard to every final program, so no configuration can gain correctness by cheating.

F. Stagnation monitor

In early development runs the loop often got stuck. The model resubmitted an identical program, or produced new programs that failed in exactly the same way. The monitor fingerprints each failure (class, diagnostic kinds and lines, failing test indices) and, when an identical program is resubmitted or the same fingerprint appears twice in a row, switches the next call to a *rewrite*: the prompt asks for a fresh approach and the temperature is raised from 0.2 to 0.6.

G. Observability

Each run writes a trajectory with state transitions, every LLM call (tokens, the model's analysis and strategy, the program), every tool call (input hash, status, diagnostics, time), the routed feedback and its class, and each stop or strategy decision. The oracle also gives a verdict for every candidate, and these per-candidate verdicts let us compute correctness after k attempts for every $k \leq K$ from a single run.

V. EXPERIMENTAL SETUP

Benchmark. We rebuilt the Phase 1 benchmark from the base paper's own data. VERINA-BASIC has 108 tasks, all of which the VERINA authors translated into Lean from Dafny originals in CloverBench (59) and MBPP-DFY (49). We kept every task whose types have a direct counterpart in both Dafny and Python (integers, naturals, booleans, characters, strings, lists and tuples), which left 97 tasks after removing ten with floats, options, maps, bytes, function arguments or nested arrays, and one malformed record. For each remaining task we wrote a Dafny specification, a Python/iContract specification and a reference solution in each language, using VERINA's Lean pre- and postcondition as the ground truth for what the task means and VERINA's positive tests as our tests (2 to 12 per task, median 5). A task was admitted only if, in both languages, (i) the reference passes the integrity guard, (ii) it verifies, (iii) it passes all tests, (iv) the empty stub is rejected, and (v) the specification accepts the expected output and rejects every known-wrong output that VERINA lists for each test input. We call check (v) a *spec probe*. In

Dafny, a probe is a copy of the method with the test input pinned by extra `requires` clauses and a body that returns the candidate output. Because both specifications answer the same concrete questions, the probes are our main evidence that the two versions of a task are equivalent. As a second check we ran both references on random inputs that satisfy the precondition and compared their outputs (differential testing). All 97 tasks passed the five checks in both languages; for three Dafny tasks the verifier could not decide some accept-probes on literal inputs, and these exemptions are documented in the task files. Differential testing on 3,698 random inputs found no disagreement between the Dafny and Python references.

Splits. Before any experiment we drew 20 tasks at random (seed 2026) as a development split and kept the remaining 77 as the test split. All design decisions (five development iterations, recorded in the repository's development log), the prompts, and the two worked examples used by E2 come from the development split only. Every number in Sections VII–IX is on the test split, which gives 154 task instances (77 per language).

Model and decoding. All experiments use `gpt-4o-mini-2024-07-18`, one of the models VERINA evaluates, which makes the base-paper comparison like for like. We had planned to add GPT-4.1, but the project had a hard budget of 2 USD for API calls, so the stronger model could not be run at the scale we needed (Section X). Agents use temperature 0.2 (0.6 for rewrites), at most 4096 output tokens, OpenAI seeds 0, 1 and 2 for the three repetitions, and a budget of $K = 5$ candidates. Every request and response is stored in a cache keyed on the full request, so all runs can be replayed exactly without new API calls.

Experiments. E1 runs the Phase 1 baselines (single-shot and testing-only). E2 reproduces VERINA's CodeGen protocol on our tasks, with the same instruction structure (task description plus a template that contains the reference specification), two worked examples, temperature 1.0 and plain-text answers. Because of the budget we drew 3 samples per task instead of VERINA's 5, but pass@1 is still estimated without bias as the mean over samples [30]. E3 runs the full SpecLoop agent, and E4 adds the verifier-only arm and reuses E1 and E3 for the gap-wise tests. In E5 we remove one component at a time from SpecLoop. Finally, E6 collects cost, latency, tool usage and reliability measures from all runs, together with a stress test of the integrity guard.

Implementation. SpecLoop is about 4,200 lines of Python. Experiments ran on a 16-core Windows 10 machine with Dafny 4.11.0, CrossHair 0.0.113 and Python 3.12, with up to 6 runs in parallel per experiment. The code, task files, raw results, trajectories and the scripts that produce every table and figure are in the project repository.

VI. Evaluation Methodology

Metrics. Our primary metric is *correctness*, the share of runs whose final program satisfies Eq. (1). We also report the *verified rate* (the verifier accepts the final program), the *test pass rate* on all tests, VERINA's CodeGen metric (pass@1 on tests, with the share of samples that do not compile), and the *spec-intact rate*. For reliability we report the *false-success rate*, the share of runs where the agent stopped because its own checks passed but the oracle disagrees (possible when a testing-only agent never sees the verifier, or when the in-loop guard is off). For efficiency we count model calls, tokens, cost in USD, verifier and test calls, and wall-clock time per run. Per-candidate oracle verdicts give *correct@*k, the correctness an agent would reach with a budget of k candidates.

Statistics. Each configuration is run three times, and we report the mean and standard deviation over the three runs. For comparisons between configurations we pair results by task: we average each task's outcome over seeds and apply a Wilcoxon signed-rank test [31] to the per-task differences, and as a second check an exact McNemar test [32] on matched (task, seed) outcomes. We also give 95% bootstrap confidence intervals for the difference in rates. For Gap 1 the pairing is by VERINA task across the two languages. We use $\alpha = 0.05$ and do not correct for multiple comparisons, so p-values near the threshold should be read with care.

Base-paper comparison. We compare against VERINA in two ways and keep them apart in every table. *Reported* numbers are VERINA's own GPT-4o-mini results on VERINA-BASIC in Lean (Table 9 of [2]: 44.0% pass@1, 54.9% of samples do not compile). *Reproduced* numbers come from E2, which runs VERINA's protocol with the same model on our Dafny and Python versions of the same tasks. The difference between the two isolates the effect of the specification language, and the difference between E2 and our agents isolates the effect of the feedback loop.

TABLE III
Main results on the test split (GPT-4o-mini, 77 tasks per language, 3 seeds). Correct = spec intact, verified and all tests pass (mean ± s.d. over seeds, %). Ver = verified, Tests = passes all tests (visible and held-out).

	Dafny			Python/iContract		
Configuration	Correct	Ver	Tests	Correct	Ver	Tests
Single-shot (E1)	46.8±1.3	46.8	64.5	96.5±0.7	96.5	98.7
Testing-only (E1)	56.3±2.7	56.3	83.1	97.8±0.7	97.8	100.0
Verifier-only (E4)	63.2±2.0	63.2	77.1	98.7±0.0	98.7	100.0
SpecLoop (E3)	**68.0**±2.7	68.0	81.8	98.7±0.0	98.7	100.0

VII. Results

Table III gives the main results on the 77 test tasks in each language. Every configuration starts from the same first candidate for a given task and seed, so the single-shot row is also the starting point of every loop.

A. Comparison with the base paper (E1, E2)

Table IV puts VERINA's reported numbers next to our reproduction of its protocol. With the same model and the same tasks, moving from Lean to Dafny raises the share of samples that pass all tests from 44.0% to 69.3%, and moving to Python raises it to 98.7%. Most of the difference is compilation: 54.9% of VERINA's GPT-4o-mini programs

TABLE IV
BASE PAPER VS. REPRODUCTION (GPT-4O-MINI, SINGLE ATTEMPT). VERINA NUMBERS ARE *reported* (LEAN, TABLE 9 OF [2]); DAFNY AND PYTHON ARE *reproduced* WITH VERINA'S PROTOCOL ON OUR VERSIONS OF THE SAME TASKS (E2, 3 SAMPLES PER TASK).

Setting	Pass@1	No compile	Verified@1	Correct@1
VERINA, Lean (reported)	44.0	54.9	n/a	n/a
E2, Dafny (reproduced)	69.3	26.4	49.8	49.8
E2, Python (reproduced)	98.7	0.0	97.0	94.8

do not compile in Lean, against 26.4% in Dafny and 0.0% in Python. VERINA's CodeGen metric does not ask for a proof, though. Once we also require the program to verify, Dafny falls to 49.8%, which is close to the Lean pass rate, while Python stays at 94.8%. Our own single-shot prompt (temperature 0.2, no worked examples) gives 46.8% on Dafny, in the same range as the 2-shot protocol.

B. RQ1: specification language (Gap 1)

The language effect is larger than any feedback effect we measured. Paired by task, Python/iContract is ahead of Dafny in every configuration, from -49.8 pp for single-shot generation (95% CI [-60.6, -39.0], Wilcoxon $p < 0.001$) to -30.7 pp for SpecLoop ($p < 0.001$). Out of 231 matched task-seed pairs under SpecLoop, Dafny is correct where Python is not in only 3 cases, and the reverse happens 74 times.

H1a predicted the opposite of what we found: Dafny's verification rate is lower, not higher (68.0% against 98.7% under SpecLoop). H1b holds: the first candidate passes all tests for 64.5% of Dafny instances and 98.7% of Python instances ($p < 0.001$). Two things drive the gap. Dafny programs fail to compile far more often (Table IV), and even a Dafny program that computes the right answers still needs loop invariants strong enough for the verifier. Under testing-only feedback 83.1% of Dafny programs pass every test but only 56.3% verify. In Python the two rates are almost the same, partly because CrossHair gives a weaker, bounded guarantee (Section X). For a team that only needs contracts checked to this standard, Python with iContract is the far easier target. For a team that needs a proof, Dafny delivers one for roughly two thirds of these small tasks after feedback, which is still well above what VERINA reports for proofs in Lean.

C. RQ2: combining verifier and test feedback (Gap 2)

On Dafny, SpecLoop is the best configuration. It is +4.8 pp above the verifier-only agent (95% CI [+0.9, +9.1]; McNemar $p = 0.043$, Wilcoxon $p = 0.081$) and +11.7 pp above the testing-only agent ($p = 0.002$). Against the verifier-only agent it wins on 18 task-seed pairs and loses on 7. The gain over the better single-source agent is just under the 5 pp we set in H2, and the two tests disagree on significance at $\alpha = 0.05$. We therefore count H2 as partly supported. On Python all three loops sit at the ceiling and the differences are zero or within one task.

The verifier-only runs show why tests help. Of the 486 failed Dafny verifications in those runs, 191 (39.3%) were on programs that already passed every test, visible and held-out. For these programs the verifier's message (usually "a postcondition could not be proved") was correct but misleading: the code was right and only the proof was missing, yet the agent had no way to know that. SpecLoop's router uses the visible tests to label such cases PROOF_GAP and asks for stronger invariants instead of a new algorithm. This is also why the combined loop needs fewer model calls than the verifier-only loop (2.65 vs. 2.74 per run).

D. RQ3: what the verifier adds over tests (Gap 3)

The testing-only agent is the baseline VERINA does not have. On Dafny it improves on single-shot generation (56.3% against 46.8%), which shows that test feedback alone is useful. Adding the verifier channel raises correctness by a further +6.9 pp (verifier-only against testing-only, 95% CI [+2.6, +11.7], $p = 0.008$), and by +11.7 pp when tests are kept as well (SpecLoop against testing-only). The gain is significant but smaller than the 10 pp we predicted in H3, so H3 is partly supported.

The more important difference is in what the agents believe when they stop. The testing-only agent stops as soon as its visible tests pass, and in 26.8% of its Dafny runs (67 runs over both languages) it stopped with a program that the oracle rejects. In every one of these runs the program passed all tests, including the held-out ones, but did not verify, so the agent had no signal that anything was missing. No run of an agent with verifier feedback ended this way. In other words, a test-driven agent cannot tell the user how reliable its answer is, while an agent that sees the verifier can. With only one model in our budget (Section X), we could not test whether this gain changes with model strength.

E. Gap-resolution summary

Table V summarises the three gaps.

VIII. ABLATION AND COMPARATIVE ANALYSIS

Ablation (E5). Table VI removes one component at a time from SpecLoop on Dafny. Python is left out because the shared first candidate already solves about 96% of the Python instances, so no component can change the outcome there. When SpecLoop's first candidate passes every check, every combined-mode variant stops at that same candidate with the same verdict. We therefore executed each variant only on the units where SpecLoop needed feedback (122 of the 231 Dafny units per variant) and reused the identical first-attempt result for the rest. Removing the test channel gives the verifier-only agent and removing the verifier channel gives the testing-only agent, so those two rows come from E4 and E1.

The verifier channel is the most important single part: without it correctness falls by -11.7 pp ($p = 0.002$). The diagnostic normaliser comes next. Replacing its structured messages with raw Dafny output costs -6.9 pp ($p = 0.010$), and removing the fix hints and loop checks added in development

TABLE V
GAP-RESOLUTION TABLE (DAFNY UNLESS NOTED; PYTHON IS AT CEILING FOR EVERY CONFIGURATION). Δ IN PERCENTAGE POINTS WITH 95% BOOTSTRAP CI AND WILCOXON p OVER TASKS.

Gap	Method / component	Metric	Base paper result	Our result	Addressed?
Gap 1	Same tasks in Dafny and Python/iContract (probe-checked specs); multi-backend verification with the diagnostic normaliser	CodeGen pass@1; correctness	Lean: 44.0% pass@1, 54.9% do not compile (reported)	Pass@1 Dafny 69.3%, Python 98.7%; SpecLoop correct Dafny 68.0%, Python 98.7% (Δ -30.7, $p < 0.001$)	Fully (H1a refuted, H1b supported)
Gap 2	Test channel + feedback router (LOGIC_BUG vs. PROOF_GAP)	Correctness vs. best single source	Not studied (single-shot only)	SpecLoop 68.0% vs. verifier-only 63.2% (Δ +4.8 [+0.9, +9.1], McNemar $p = 0.043$)	Partially (gain below 5 pp target)
Gap 3	Verifier feedback channel vs. identical testing-only agent	Correctness gain; false-success rate	No testing-only baseline	Δ +6.9 [+2.6, +11.7], $p = 0.008$; false success 26.8% vs. 0%	Partially (gain below 10 pp target; one model)

TABLE VI
ABLATION ON DAFNY (E5): CORRECTNESS WHEN ONE COMPONENT IS REMOVED (MEAN ± S.D. OVER 3 SEEDS, %). Δ IS THE PAIRED DIFFERENCE TO THE FULL SYSTEM IN PERCENTAGE POINTS WITH 95% BOOTSTRAP CI.

Variant	Correct	Δ [95% CI]	p
Full SpecLoop	68.0±2.7	–	–
– normaliser (raw output)	61.0±1.3	-6.9 [-11.7, -2.6]	= 0.010
– router (plain concatenation)	64.9±2.2	-3.0 [-6.5, +0.4]	= 0.251
– guard in the loop	68.0±2.7	+0.0 [+0.0, +0.0]	= 1.000
– stagnation monitor	66.7±3.3	-1.3 [-3.0, +0.0]	= 0.102
– dev fixes (iterations 4–5)	61.5±1.5	-6.5 [-10.8, -2.6]	= 0.011
– test channel (= verifier-only)	63.2±2.0	-4.8 [-9.1, -0.9]	= 0.081
– verifier channel (= testing-only)	56.3±2.7	-11.7 [-17.7, -6.1]	= 0.002

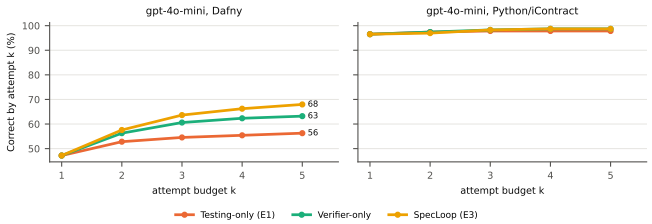


Fig. 2. Correctness after k candidates (from per-candidate oracle verdicts). All configurations share the first candidate, so they start from the same point at $k = 1$.

iterations 4 and 5 costs -6.5 pp ($p = 0.011$). Neither effect was visible on the 20 development tasks with one seed (Section X), so the test split confirms decisions that the dev split could only suggest. Removing the test channel costs -4.8 pp and removing the router -3.0 pp; both point in the expected direction, but neither is significant on its own, which matches the modest RQ2 effect. The stagnation monitor adds about one point (-1.3 pp, $p = 0.102$). Switching the in-loop guard off changes nothing at all, because GPT-4o-mini almost never edited the specification in these runs; the guard matters for validity, not for accuracy (see below).

Attempt budget. Fig. 2 shows correctness as a function of the number of candidates k. On Dafny all three loops start from the same point (47.2% at $k = 1$) and separate after the first round of feedback. The testing-only curve is almost flat after $k = 2$ (52.8% to 56.3%), since once the visible tests pass it has nothing left to fix. The verifier-only curve rises to 63.2%, and SpecLoop rises fastest and is still gaining at $k = 5$ (66.2% to 68.0%), so a larger budget would probably help it more than the others. Across three seeds SpecLoop turned 48 failing first candidates into correct Dafny programs, against 37 for verifier-only and 21 for testing-only.

Efficiency (E6). On Dafny a run costs 0.21 m$ (thousandths of a dollar) for single-shot generation, 0.52 m$ for testing-only, 0.79 m$ for verifier-only and 0.78 m$ for SpecLoop, with 1.00, 1.97, 2.74 and 2.65 model calls per run on average. The cost that matters, though, is the cost per correct program. There, SpecLoop (1.20 m$) is cheaper than the verifier-only agent (1.20 m$), because the router lets it solve more tasks with slightly fewer calls. The testing-only agent is cheaper still per solved task (0.90 m$), but it reaches a lower ceiling and, as shown above, cannot tell when its answer is unverified. Verification dominates the latency of the verifier-based loops: tool time is 66.3% of SpecLoop's model-plus-tool time, against 32.3% for testing-only. On Python every configuration costs about the same, since nearly all runs end after one candidate. The whole study, including development runs and E2, cost 1.04 USD in API calls.

Reliability and the integrity guard (E6). To test whether "verified" can be trusted, we built tampered versions of every reference solution: a dropped or weakened `ensures`, an added `requires false`, `assume false`, `{:verify false}`, an axiom lemma, an edited helper predicate, a removed or weakened iContract decorator, an added `require(lambda ...: False)`, a CrossHair-off comment, and a patched `icontract`. We also built harmless variants that only reformat or add comments. The guard rejected all 591 Dafny and all 485 Python tampering mutants and none of the 194 plus 194 harmless ones. Without the guard the result would have looked very different: Dafny alone accepted 529 of the 591 tampered programs, and CrossHair accepted 100 of the 100 Python mutants we ran through it. In the actual runs GPT-4o-mini rarely tried to change the specification (the in-loop guard fired in 1 SpecLoop run), so

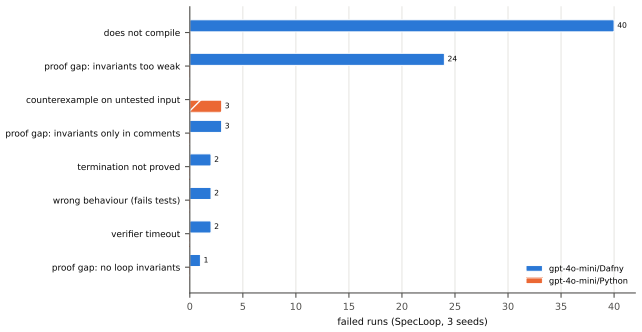


Fig. 3. Failure categories of the full SpecLoop agent on the test split (all failed runs over three seeds).

the guard is mostly insurance, but it is what lets us report a verified program as correct. The stagnation monitor switched to a rewrite in 60 SpecLoop runs, and 9 of those ended correct.

IX. Failure Analysis

We took every run of the full system that ended without a correct program and assigned it to one category, using the last oracle verdict, the last verifier diagnostics and the final program. The categories and their frequencies are shown in Fig. 3.

Over three seeds the full agent failed 74 Dafny runs and 3 Python runs. Four categories cover almost all of them.

Does not compile (40 Dafny runs, 54.1%). The most frequent failure is a program that never compiles, even after four repairs. The errors are the ones that motivated development iteration 4: array operations on sequences, string building with constructs Dafny does not have, statement `if` written with `then`, and calls to library functions that do not exist. The fix hints help (vb_017, vb_082 and vb_105 were solved on the dev split only after they were added), but GPT-4o-mini often swaps one compile error for another instead of converging. This mirrors VERINA, where most GPT-4o-mini failures were Lean compile errors. Better language grounding, for example retrieving short Dafny idioms for the construct that failed, is the obvious next step.

Proof gap (24 + 3 + 1 runs). The program passes every test but does not verify. In most of these runs the invariants exist but are too weak to imply the postcondition, typically a bound on the index without a clause that relates the partial result to the specification. A few runs still write the invariants as comments or leave them out entirely. Here the router does its job and labels these attempts PROOF_GAP, but the model does not know how to strengthen the invariant. Invariant-generation techniques such as those in Lemur [17] or Laurel [16] would slot into exactly this branch of the loop.

Termination and timeouts (2 + 2 runs). Missing `decreases` clauses and quantifier-heavy invariants that exhaust the verifier's time limit are rare with these small tasks.

Wrong behaviour and counterexamples (2 Dafny, 3 Python runs). Very few runs end with code that is actually wrong. In Python every failure is a program that passes all tests but for which CrossHair finds an input that breaks a contract, which is precisely the kind of error the verifier is there to catch.

Representative cases. The trajectories are in `results/cases/` of the repository. *Success 1* (vb_016, Dafny, replace characters): the first candidate does not compile (array syntax on a sequence) and is routed as SYNTAX. The second compiles and passes every visible test but does not verify, so the router labels it PROOF_GAP. The model keeps the algorithm, adds an invariant that ties the partial result to the input, and the third candidate verifies. *Success 2* (vb_033, Python, smallest missing number): the first candidate passes all five tests, but CrossHair finds a sorted input with duplicates that breaks the postcondition (LOGIC_BUG). The next candidate fails in the same way, the stagnation monitor switches to a rewrite, and the rewritten set-based version is correct. A testing-only agent would have stopped at the first, wrong program. *Failure 1* (vb_009, Dafny, common element): the code is right from the first attempt and passes all eight tests, but all five candidates fail on invariants about the multiset of seen elements; each repair changes the invariants without making them strong enough. *Failure 2* (vb_086, Dafny, rotate a sequence): the first candidate tries to fill an array where the specification works with sequences, the second and third use a comprehension syntax that does not exist in Dafny, and the fourth goes back to a resolution error. All four are routed as SYNTAX. The fifth candidate finally compiles but rotates in the wrong direction and passes only one of six tests.

X. Limitations and Threats to Validity

One model. The API budget for the whole project was 2 USD, so every experiment uses GPT-4o-mini. This keeps the base-paper comparison exact, since VERINA reports the same checkpoint, but it leaves RQ3's secondary question (does the gain hold across models) open. A stronger model might make fewer compile errors and more proof gaps, which would shift the weight between components. The agent works with any OpenAI-compatible endpoint, and the cache makes adding a model cheap.

Unequal verifiers. Dafny proves the postcondition for all inputs, while CrossHair only searches for a counterexample for a fixed time per condition, a weaker guarantee. Its time-based search also explores less on a loaded machine, and our runs used all 16 cores. Part of the RQ1 gap is therefore a gap between guarantees, which is why we report the test pass rate next to the verified rate throughout.

Specification equivalence. Shared tests, spec probes and differential testing support the equivalence of each Dafny/Python pair but do not prove it. Some Dafny specifications were reformulated so that the verifier could decide the probes, three tasks keep a documented probe exemption, and a few tasks fix an output length that VERINA's Lean postcondition left open.

Tasks. VERINA-BASIC tasks are small and drawn from MBPP and textbook algorithms, so they are likely in the

training data. Python is near the ceiling partly for this reason, which leaves little room for differences in its half of RQ2 and RQ3 and is why E5 runs on Dafny only. VERINA-ADV was out of scope because most of its types have no direct Python/iContract counterpart.

Statistics and design decisions. Three seeds at temperature 0.2 give limited variance estimates, and we do not correct for multiple comparisons. Configurations share their first candidate, which lowers variance but means differences come only from tasks the first attempt does not solve. Development decisions rested on 20 tasks and one seed, where each change moved one or two tasks; E5 re-measures every kept component on the test split.

Base-paper comparison. E2 reproduces VERINA's protocol, not its code: prompts are rewritten for Dafny and Python, we draw 3 samples per task instead of 5, the worked examples come from our development split, and VERINA's Lean numbers are taken from the paper.

XI. Conclusion

We set out to answer three questions that VERINA leaves open. On the same 97 tasks and with the same model, the specification language turned out to matter most. Moving from Lean to Dafny to Python/iContract raises the share of programs that pass their tests from 44.0% to 69.3% to 98.7%, and Python stays well ahead of Dafny inside every agent loop, although part of that lead comes from CrossHair's weaker, bounded guarantee. Verifier feedback improves on a testing-only agent by +6.9 pp on Dafny. Its larger benefit is that the agent no longer stops early: the testing-only agent declared success on unverified code in 26.8% of its Dafny runs, and no verifier-based agent did. Combining the two signals works best. Tests show that a large share of verifier failures (39.3% in our runs) are missing proofs rather than wrong code, and SpecLoop, which uses that information, is the most accurate configuration, although the margin over verifier-only feedback (+4.8 pp) is smaller than we hypothesised.

The remaining failures point to clear next steps. Most failed Dafny runs never compile, so the agent needs better grounding in the language, for example retrieved idioms for the construct that failed. Most of the rest are proof gaps where the invariant is too weak, which is where dedicated invariant generators could be plugged into the router's PROOF_GAP branch. We would also like to repeat the study with stronger and open-weight models, which the budget did not allow here, and on the harder VERINA-ADV tasks. Because every model response is cached, adding a model or a task only costs the new calls.

References

[1] Z. Wang *et al.*, "Towards understanding the characteristics of code generation errors made by large language models," in *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE, 2025, pp. 2587–2599. [Online]. Available: https://doi.org/10.1109/icse55347.2025.00180

[2] Z. Ye *et al.*, "VERINA: Benchmarking verifiable code generation," in *International Conference on Learning Representations (ICLR)*, 2026. [Online]. Available: https://openreview.net/forum?id=0A4Uf88pog

[3] P. Aggarwal, B. Parno, and S. Welleck, "AlphaVerus: Bootstrapping formally verified code generation through self-improving translation and treefinement," in *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 267. PMLR, 2025, pp. 587–615. [Online]. Available: https://proceedings.mlr.press/v267/aggarwal25a.html

[4] C. Yang *et al.*, "AutoVerus: Automated proof generation for Rust code," *Proceedings of the ACM on Programming Languages*, vol. 9, no. OOPSLA2, pp. 3454–3482, 2025. [Online]. Available: https://doi.org/10.1145/3763174

[5] C. R. Loughridge *et al.*, "DafnyBench: A benchmark for formal software verification," *Transactions on Machine Learning Research*, 2025. [Online]. Available: https://openreview.net/forum?id=yBgTVWccIx

[6] K. R. M. Leino, "Dafny: An automatic program verifier for functional correctness," in *Logic for Programming, Artificial Intelligence, and Reasoning (LPAR 2010)*, ser. Lecture Notes in Computer Science, vol. 6355. Berlin, Heidelberg: Springer, 2010, pp. 348–370. [Online]. Available: https://doi.org/10.1007/978-3-642-17511-4_20

[7] P. Schanely, "CrossHair: Analyze Python code for correctness using symbolic execution," GitHub repository, 2026, python package `crosshair-tool`, version 0.0.113. Accessed Nov. 2026. [Online]. Available: https://github.com/pschanely/CrossHair

[8] A. Thakur *et al.*, "CLEVER: A curated benchmark for formally verified code generation," in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, vol. 38, 2025, pp. 128 165–128 199. [Online]. Available: https://doi.org/10.52202/085713-3857

[9] M. R. H. Misu *et al.*, "Towards AI-assisted synthesis of verified Dafny methods," *Proceedings of the ACM on Software Engineering*, vol. 1, no. FSE, pp. 812–835, 2024. [Online]. Available: https://doi.org/10.1145/3643763

[10] A. Shefer *et al.*, "Can LLMs enable verification in mainstream programming?" arXiv preprint arXiv:2503.14183, 2025. [Online]. Available: https://arxiv.org/abs/2503.14183

[11] X. Chen *et al.*, "Teaching large language models to self-debug," in *International Conference on Learning Representations (ICLR)*, 2024. [Online]. Available: https://openreview.net/forum?id=KuPixIqPiq

[12] N. Shinn *et al.*, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023, pp. 8634–8652. [Online]. Available: https://doi.org/10.52202/075280-0377

[13] L. Zhong, Z. Wang, and J. Shang, "Debug like a human: A large language model debugger via verifying runtime execution step by step," in *Findings of the Association for Computational Linguistics: ACL 2024*. Bangkok, Thailand: Association for Computational Linguistics, 2024, pp. 851–870. [Online]. Available: https://doi.org/10.18653/v1/2024.findings-acl.49

[14] C. Sun *et al.*, "Clover: Closed-loop verifiable code generation," in *AI Verification (SAIV 2024)*, ser. Lecture Notes in Computer Science, vol. 14846. Cham: Springer, 2024, pp. 134–155. [Online]. Available: https://doi.org/10.1007/978-3-031-65112-0_7

[15] T. Chen *et al.*, "Automated proof generation for Rust code via self-evolution," in *International Conference on Learning Representations (ICLR)*, 2025. [Online]. Available: https://openreview.net/forum?id=2NqssmiXLu

[16] E. Mugnier *et al.*, "Laurel: Unblocking automated verification with large language models," *Proceedings of the ACM on Programming Languages*, vol. 9, no. OOPSLA1, pp. 1519–1545, 2025. [Online]. Available: https://doi.org/10.1145/3720499

[17] H. Wu, C. Barrett, and N. Narodytska, "Lemur: Integrating large language models in automated program verification," in *International Conference on Learning Representations (ICLR)*, 2024. [Online]. Available: https://openreview.net/forum?id=Q3YaCghZNt

[18] E. First *et al.*, "Baldur: Whole-proof generation and repair with large language models," in *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. San Francisco, CA, USA: ACM, 2023, pp. 1229–1241. [Online]. Available: https://doi.org/10.1145/3611643.3616243

[19] A. Thakur *et al.*, "An in-context learning agent for formal theorem-proving," in *Conference on Language Modeling (COLM)*, 2024. [Online]. Available: https://openreview.net/forum?id=V7HRrxXUhN

[20] S. Chakraborty *et al.*, "Towards neural synthesis for SMT-assisted proof-oriented programming," in *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE,

2025, pp. 1755–1767. [Online]. Available: https://doi.org/10.1109/icse55347.2025.00002

[21] L. Ma *et al.*, "SpecGen: Automated generation of formal program specifications via large language models," in *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE, 2025, pp. 16–28. [Online]. Available: https://doi.org/10.1109/icse55347.2025.00129

[22] C. Wen *et al.*, "Enchanting program specification synthesis by large language models using static analysis and program verification," in *Computer Aided Verification (CAV 2024)*, ser. Lecture Notes in Computer Science, vol. 14682. Cham: Springer, 2024, pp. 302–328. [Online]. Available: https://doi.org/10.1007/978-3-031-65630-9_16

[23] M. Endres *et al.*, "Can large language models transform natural language intent into formal method postconditions?" *Proceedings of the ACM on Software Engineering*, vol. 1, no. FSE, pp. 1889–1912, 2024. [Online]. Available: https://doi.org/10.1145/3660791

[24] S. B. Hossain and M. B. Dwyer, "TOGLL: Correct and strong test oracle generation with LLMs," in *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE, 2025, pp. 1475–1487. [Online]. Available: https://doi.org/10.1109/icse55347.2025.00098

[25] A. Madaan *et al.*, "Self-Refine: Iterative refinement with self-feedback," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023, pp. 46 534–46 594. [Online]. Available: https://doi.org/10.52202/075280-2019

[26] B. Chen *et al.*, "CodeT: Code generation with generated tests," in *International Conference on Learning Representations (ICLR)*, 2023. [Online]. Available: https://openreview.net/forum?id=ktrw68Cmu9c

[27] S. Yao *et al.*, "ReAct: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations (ICLR)*, 2023. [Online]. Available: https://openreview.net/forum?id=WE_vluYUL-X

[28] J. Yang *et al.*, "SWE-agent: Agent-computer interfaces enable automated software engineering," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 37, 2024, pp. 50 528–50 652. [Online]. Available: https://doi.org/10.52202/079017-1601

[29] L. de Moura and N. Bjørner, "Z3: An efficient SMT solver," in *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2008)*, ser. Lecture Notes in Computer Science, vol. 4963. Berlin, Heidelberg: Springer, 2008, pp. 337–340. [Online]. Available: https://doi.org/10.1007/978-3-540-78800-3_24

[30] M. Chen *et al.*, "Evaluating large language models trained on code," arXiv preprint arXiv:2107.03374, 2021. [Online]. Available: https://arxiv.org/abs/2107.03374

[31] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bulletin*, vol. 1, no. 6, 1945. [Online]. Available: https://doi.org/10.2307/3001968

[32] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947. [Online]. Available: https://doi.org/10.1007/BF02295996